

计算机组成

单周期控制

(2025秋)

高小鹏

北京航空航天大学计算机学院

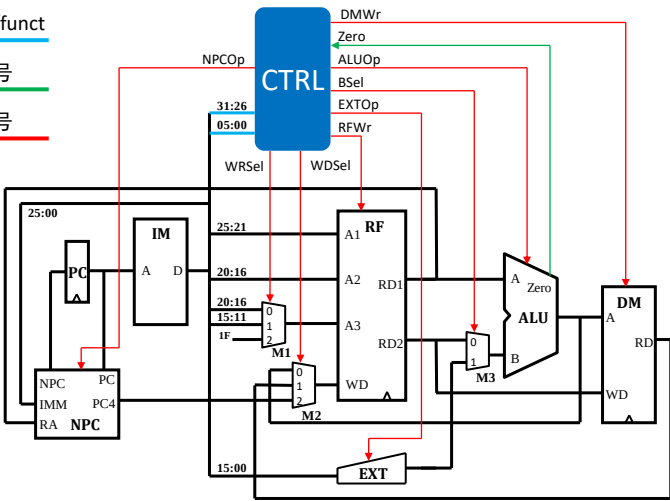
数据通路与控制器

- 控制器的职责：根据各指令的opcode（指令的31:26位）和funct（指令的05:00位），产生各功能部件的控制信号表达式

输入：opcode/funct

输入：状态信号

输出：控制信号



计算机组成与实现

合成各指令的控制信号取值矩阵

□ 示例：addu指令

环节	步骤	语义	连接关系	功能部件	控制信号取值
1	读指令	读取指令 计算下条指令地址 更新PC	$\langle PC.DO, IM.A \rangle$ $\langle PC.DO, NPC.PC \rangle$ $\langle NPC.NPC, PC.DI \rangle$	IM PC NPC	NPCOp: PC+4
2	读操作数		$\langle IM.D[25:21], RF.A1 \rangle$ $\langle IM.D[20:16], RF.A2 \rangle$	RF	
3	执行	ALU执行加法	$\langle RF.RD1, ALU.A \rangle$ $\langle RF.RD2, ALU.B \rangle$	ALU	ALUOp: ADD
4	回写	计算结果回写至rd寄存器	$\langle ALU.C, RF.WD \rangle$ $\langle IM.D[15:11], RF.A3 \rangle$	RF	RFWr: 1

- 对于不涉及的组合逻辑功能部件，如EXT，其控制信号可以设置为X
 - X有助于化简。一般来说，令其为0会得到更简单的控制信号表达式
- 对于不涉及的存储功能部件，如DM，其写使能必须设置为0
 - 如果DM的写使能不设置为0，则意味着addu指令也会导致DM被写入

指令	NPCOp	RFWr	EXTOp	ALUOp	DMWr	M1Sel	M2Sel	M3Sel
addu	PC+4	1	X	ADD	0	?	?	?

计算机组成与实现

控制信号取值矩阵

□ 将各条指令的控制信号取值汇总为一个完整的控制信号取值矩阵

- 注意：由于单条指令的建模过程中不涉及MUX，因此矩阵不包含MUX的控制信号的取值

指令	NPCOp	RFWr	EXTOp	ALUOp	DMWr	M1Sel	M2Sel	M3Sel
addu	PC+4	1	X	ADD	0	?	?	?
subu	PC+4	1	X	SUB	0	?	?	?
ori	PC+4	1	0	OR	0	?	?	?
lw	PC+4	1	1	ADD	0	?	?	?
sw	PC+4	0	1	ADD	1	?	?	?
beq	0b01	0	X	SUB	0	?	?	?
jal	0b10	1	X	X	0	?	?	?
jr	0b11	0	X	X	0	?	?	?

计算机组成与实现

通过反查方法构造MUX控制信号真值表

- 根据指令与输入信号关系及端口与输入源关系，就可以“自动”确定控制信号表达式
- 对于RF.A3的MUX，以sw指令为例
 - 由于lw指令的20:16位是RF的回写编号，由此其对应1号端口
 - 这意味着为了执行lw指令，RF.A3的MUX的控制信号取值必须为1（或者0b01）

指令与RF.A3		RF.A3的MUX		MUX控制信号真值表		
指令	RF.A3	端口编号	输入源	指令	端口编号	控制信号取值
addu	IM.D[15:11]	0	IM.D[15:11]	add	0端口	0b00
subu	IM.D[15:11]	1	IM.D[20:16]	sub	0端口	0b00
ori	IM.D[20:16]	2	0x1F	ori	1端口	0b01
lw	IM.D[20:16]			lw	1端口	0b01
sw				sw		
beq				beq		
jal	0x1F			jal	2端口	0b10
jr				jr		

计算机组成与实现

完整的控制信号取值矩阵（包含MUX的控制信号）

- 按照前述方法可以构造出所有MUX的控制信号取值
 - 注意：NPCOp/ALUOp，采用了宏定义；相比M1Sel/M2Sel等，宏定义更易于理解、有利于代码维护

指令	NPCOp	RFWr	EXTOp	ALUOp	DMWr	M1Sel	M2Sel	M3Sel
addu	PC+4	1	X	ADD	0	00	00	0
subu	PC+4	1	X	SUB	0	00	00	0
ori	PC+4	1	0	OR	0	XX	00	1
lw	PC+4	1	1	ADD	0	XX	01	1
sw	PC+4	0	1	ADD	1	XX	XX	1
beq	0b01	0	X	SUB	0	XX	XX	0
jal	0b10	1	X	X	0	XX	10	X
jr	0b11	0	X	X	0	XX	XX	X

- Q：该如何将这个表格转换为组合逻辑表达式？

计算机组成与实现

使用指令的opcode与funct产生变量（VerilogHDL版）

- 每条指令设置一个指令变量
 - ◆ 为了提高可读性，变量名就是指令名（例如变量beq对应beq指令）
- 由于R型指令的opcode都为0，因此需产生一个单独变量Rtype
- 示例：Verilog表达式

指令	opcode	funct
...	...	...
addu	000000	100000
beq	000100	
...	...	...

```
assign beq = (op==`BEQ)
assign Rtype = (op==6'b000000)
assign add = Rtype&(funct==`ADDU)
```

‘BEQ和‘ADDU是宏定义
宏有助于提高可读性，
更易于工程维护

计算机组成与实现

使用指令变量构造控制信号（VerilogHDL版）

- 示例
 - DMWr = sw
 - RFWr = add + sub + ori + lw + jal
- 构造控制信号表达式时，要确保包含了所有取值为1的指令

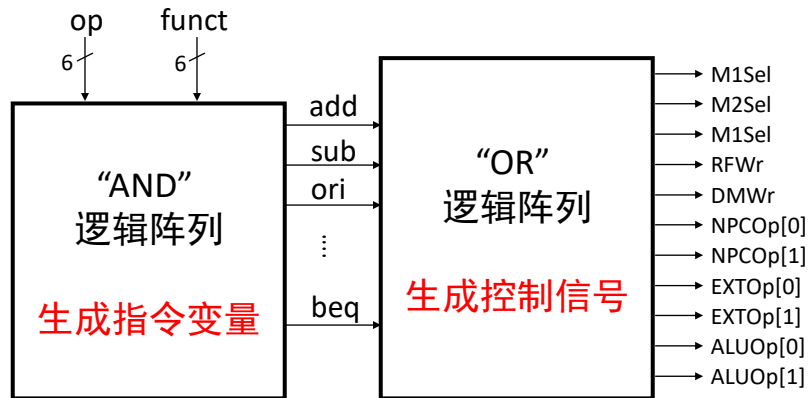
本质就是根据真值表
生成表达式的方法

指令	NPCOp	RFWr	EXTOp	ALUOp	DMWr	M1Sel	M2Sel	M3Sel
addu	PC+4	1	X	ADD	0	00	00	0
subu	PC+4	1	X	SUB	0	00	00	0
ori	PC+4	1	0	OR	0	01	00	1
lw	PC+4	1	1	ADD	0	01	01	1
sw	PC+4	0	1	ADD	1	XX	XX	1
beq	0b01	0	X	SUB	0	XX	XX	0
jal	0b10	1	X	X	0	10	10	X
jr	0b11	0	X	X	0	XX	XX	X

计算机组成与实现

用AND和OR两个阵列来实现控制器（Logisim版）

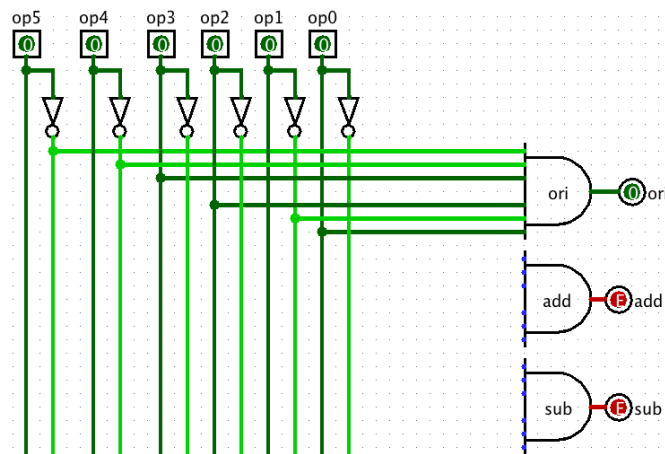
- AND阵列生成指令变量
- OR阵列生成控制信号



计算机组成与实现

AND阵列生成指令变量（Logisim版）

- 将op[5:0]的每个信号正反均引出（funct[5:0]同理处理）
- 每条指令都对应一个AND门，根据指令的opcode定义决定连接关系
- 示例：ori的opcode为001101



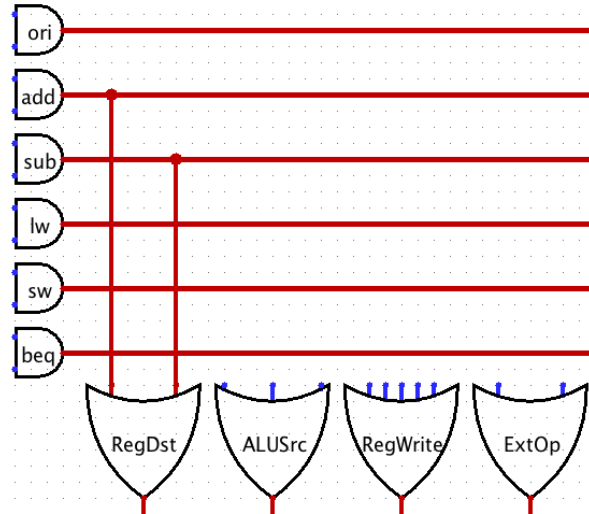
提示

对于R型指令，还需要额外构造RTYPE的电路（一个6输入AND）

计算机组成与实现

OR阵列生成控制信号（Logisim版）

- 构造思路与用VerilogHDL版是相同的



计算机组成与实现